a. Name of two GenAI programs
The 2 GenAIs that I'm using for this are Gemini and ChatGPT
b. How you accessed them (e.g., Google query, CoPilot on my computer)
I accessed both LLMs through a google search and used the main website used for both AIs.
c. Prompt used. (You must use the same prompt for both GenAIs.)
Use two GenAI's to generate C++ code using objects, not functions, to implement the following requirements (Note: You may use either ChatGPT or CoPilot as one of your choices, but not both. They are essentially the same GenAI.):
• Sudoku is a number puzzle in which a 9x9 grid is subdivided into 9 3x3 sub-grids, or blocks.
• A typical puzzle is shown here (image credit: Wikimedia Commons):
o
• Each of the cells in the puzzle must be filled in with a single digit 1-9, subject to the following constraints:
o Each digit from 1 to 9 must appear exactly once in each row.
o Each digit from 1 to 9 must appear exactly once in each column.
o Each digit from 1 to 9 must appear exactly once in each block.
• Published puzzles are usually constructed to have a unique solution. This requires that at least 17 cells be filled in (and may require more; depending on the puzzle and which cells are empty, as many as 78 filled cells can be required to guarantee uniqueness). In general, the number of cells that are already filled in is not a particularly good indicator of how difficult a Sudoku puzzle is to solve.
• The Sudoku problem, then, is this: Given a partially completed Sudoku puzzle, find the (or at least a) solution; if no solution exists, prove that.
• It is possible to solve a Sudoku puzzle entirely through logic. For example, in the above puzzle, consider the bottom right block, top right cell. Because of other numbers in the same row, column, or grid, that number cannot be 1, 2, 3, 5, 6, 7, 8, or 9, and so must be 4. By proceeding in this fashion, using these and more advanced deduction rules, it's possible to gradually fill in the entire grid.
• That is not the approach we're going to use. Instead, we're going to implement a brute-force search, in which we exhaustively try possibilities until we find a solution. This will be slower to run but is easier to solve. (And by "slower to run," we mean it'll finish in a few tenths of a second rather than a few milliseconds.)
• Here's the idea: Suppose we have a Sudoku puzzle of which k of the 81 cells are unfilled. We identify an empty cell and deduce a list of candidates for what might go into that cell. If there are no such values (all digits are accounted for elsewhere in the same row, column, or grid), then we must have made a mistake in filling in the k cells and return a no-solution-found result. Otherwise, we try putting the first candidate value into the empty cell, and see if we can find a solution with the k+1 cells filled (i.e. a recursive call). If we get a no-solution-found to the first candidate, then we try the next, and so on,

until we either find a solution (k == 81) or determine that no solution exists (all candidates report no solution, which again implies we took a wrong path somewhere earlier).
• This approach, by the way, is called depth-first search with backtracking. For each empty cell we make an attempt and see how far we can get with it. If we fail to find a solution, we back up until we find something else to try, and so on. If a solution exists, we are guaranteed to find it, because we exhaustively try all possibilities.
• You are given five files (puzzle1.txt through puzzle5.txt), each containing a Sudoku puzzle. Digits already filled in have been placed, and an underscore character (_) is used to indicate a blank square.
• Write a recursive object-oriented program to find the solution to the five puzzle files.
• The output for each solution, should include:
o The puzzle file name (e.g., puzzle1.txt)
o The puzzle from the file printed out.
o The solution to the puzzle printed out or if no solution is found, “No solution found” printed out.
• The solution may not be unique; if there is more than one solution to the puzzle, your program should print them all out.
• To help you debug your program, solution1.txt is a solution to puzzle1.txt.

d. Generated code from both GenAIs.
Gemini Code: #include <iostream>

```
#include <fstream>
#include <vector>
#include <string>

class SudokuSolver {
public:
    static const int SIZE = 9;
    static const int SUBGRID = 3;

    SudokuSolver() : solutionCount(0) {
        grid.assign(SIZE, std::vector<char>(SIZE, '_'));
    }

    // Loads puzzle grid from the specified file
    bool loadFromFile(const std::string& filename) {
        std::ifstream inFile(filename);
        if (!inFile) {
            std::cerr << "Error: Could not open file " << filename << "\n";
            return false;
        }
```

```
    std::string line;
    int row = 0;
    while (row < SIZE && inFile >> line) {
        // Read valid non-whitespace characters representing cells
        for (size_t col = 0; col < line.size() && row < SIZE; ++col) {
            char ch = line[col];
            if (ch == '_' || (ch >= '1' && ch <= '9')) {
                grid[row][col] = ch;
            }
        }
        row++;
    }

    inFile.close();
    return (row == SIZE);
}

// Prints the current state of the grid
void displayGrid() const {
    for (int r = 0; r < SIZE; ++r) {
        for (int c = 0; c < SIZE; ++c) {
            std::cout << grid[r][c] << (c == SIZE - 1 ? "" : " ");
        }
        std::cout << "\n";
    }
}

// Executes the recursive depth-first search to find all solutions
void solve(const std::string& filename) {
    std::cout << "========================================\n";
    std::cout << "Puzzle File: " << filename << "\n";
    std::cout << "----------------------------------------\n";
    std::cout << "Initial Puzzle:\n";
    displayGrid();
    std::cout << "----------------------------------------\n";

    solutionCount = 0;
    backtrack(0, 0);

    if (solutionCount == 0) {
        std::cout << "No solution found\n";
    } else {
        std::cout << "Total Solutions Found: " << solutionCount << "\n";
```

```
        }
        std::cout << "========================================\n\n";
    }

private:
    std::vector<std::vector<char>> grid;
    int solutionCount;

    // Checks row, column, and 3x3 block constraints
    bool isValid(int row, int col, char num) const {
        // Check row and column
        for (int i = 0; i < SIZE; ++i) {
            if (grid[row][i] == num || grid[i][col] == num) {
                return false;
            }
        }

        // Check 3x3 block
        int startRow = (row / SUBGRID) * SUBGRID;
        int startCol = (col / SUBGRID) * SUBGRID;
        for (int r = 0; r < SUBGRID; ++r) {
            for (int c = 0; c < SUBGRID; ++c) {
                if (grid[startRow + r][startCol + c] == num) {
                    return false;
                }
            }
        }

        return true;
    }

    // Returns all candidates (1-9) that can legally be placed at (row, col)
    std::vector<char> getCandidates(int row, int col) const {
        std::vector<char> candidates;
        for (char num = '1'; num <= '9'; ++num) {
            if (isValid(row, col, num)) {
                candidates.push_back(num);
            }
        }
        return candidates;
    }

    // Recursive depth-first search with backtracking
    void backtrack(int row, int col) {
```

```
        // End of row reached: move to the next row
        if (col == SIZE) {
            row++;
            col = 0;
        }

        // Base case: all 81 cells traversed successfully
        if (row == SIZE) {
            solutionCount++;
            std::cout << "Solution #" << solutionCount << ":\n";
            displayGrid();
            std::cout << "\n";
            return; // Backtrack to find any other solutions
        }

        // Skip cells already filled with clues
        if (grid[row][col] != '_') {
            backtrack(row, col + 1);
            return;
        }

        // Deduce candidates for current empty cell
        std::vector<char> candidates = getCandidates(row, col);

        // Exhaustively try each candidate
        for (char candidate : candidates) {
            grid[row][col] = candidate;       // Place candidate
            backtrack(row, col + 1);          // Recurse to next cell
            grid[row][col] = '_';             // Backtrack / Undo choice
        }
    }
};

int main() {
    std::vector<std::string> puzzleFiles = {
        "puzzle1.txt",
        "puzzle2.txt",
        "puzzle3.txt",
        "puzzle4.txt",
        "puzzle5.txt"
    };

    for (const std::string& filename : puzzleFiles) {
        SudokuSolver solver;
```

```
        if (solver.loadFromFile(filename)) {
            solver.solve(filename);
        }
    }

    return 0;
}
```

Chat GPT Code:

```
#include <iostream>
#include <fstream>
#include <string>
#include <vector>
using namespace std;

class SudokuSolver {
private:
    int grid[9][9];

    // Checks whether a number can be placed in a cell.
    bool isValid(int row, int col, int num) {
        // Check the row and column.
        for (int i = 0; i < 9; i++) {
            if (grid[row][i] == num)
                return false;

            if (grid[i][col] == num)
                return false;
        }

        // Find the top-left corner of the 3x3 block.
        int blockRow = (row / 3) * 3;
        int blockCol = (col / 3) * 3;

        // Check the 3x3 block.
        for (int r = blockRow; r < blockRow + 3; r++) {
            for (int c = blockCol; c < blockCol + 3; c++) {
                if (grid[r][c] == num)
                    return false;
            }
        }

        return true;
```

```
}

// Recursive depth-first search with backtracking.
void solveRecursive(int row, int col,
                    vector<array<int, 81>>& solutions,
                    int& solutionCount) {
    // Stop at the end of the grid.
    if (row == 9) {
        array<int, 81> solution;
        for (int r = 0; r < 9; r++) {
            for (int c = 0; c < 9; c++) {
                solution[r * 9 + c] = grid[r][c];
            }
        }

        solutions.push_back(solution);
        solutionCount++;
        return;
    }

    // Move to the next cell.
    int nextRow = row;
    int nextCol = col + 1;

    if (nextCol == 9) {
        nextRow++;
        nextCol = 0;
    }

    // If the cell is already filled, skip it.
    if (grid[row][col] != 0) {
        solveRecursive(nextRow, nextCol,
                       solutions, solutionCount);
        return;
    }

    // Try every possible digit in the empty cell.
    for (int num = 1; num <= 9; num++) {
        if (isValid(row, col, num)) {
            grid[row][col] = num;

            solveRecursive(nextRow, nextCol,
                           solutions, solutionCount);
```

```
            // Backtrack to try another candidate.
            grid[row][col] = 0;
        }
    }
}

public:
    // Constructor initializes all cells as empty.
    SudokuSolver() {
        for (int r = 0; r < 9; r++) {
            for (int c = 0; c < 9; c++) {
                grid[r][c] = 0;
            }
        }
    }

    // Reads a puzzle from a text file.
    bool loadPuzzle(const string& filename) {
        ifstream inputFile(filename);

        if (!inputFile.is_open()) {
            cout << "Could not open " << filename << endl;
            return false;
        }

        char ch;
        int count = 0;

        while (inputFile.get(ch) && count < 81) {
            if (ch >= '1' && ch <= '9') {
                grid[count / 9][count % 9] = ch - '0';
                count++;
            }
            else if (ch == '_') {
                grid[count / 9][count % 9] = 0;
                count++;
            }
            // Ignore spaces, line breaks, and other separators.
        }

        inputFile.close();

        if (count != 81) {
            cout << "Invalid puzzle format in "
```

```
            << filename << endl;
        return false;
    }

    return true;
}

// Prints the current grid.
void printPuzzle() {
    for (int r = 0; r < 9; r++) {
        for (int c = 0; c < 9; c++) {
            if (grid[r][c] == 0)
                cout << "_ ";
            else
                cout << grid[r][c] << " ";

            if (c == 2 || c == 5)
                cout << "| ";
        }

        cout << endl;

        if (r == 2 || r == 5)
            cout << "------+-------+------" << endl;
    }
}

// Finds and prints every solution.
void solvePuzzle() {
    vector<array<int, 81>> solutions;
    int solutionCount = 0;

    solveRecursive(0, 0, solutions, solutionCount);

    if (solutionCount == 0) {
        cout << "No solution found." << endl;
        return;
    }

    cout << "Number of solutions found: "
         << solutionCount << endl;

    for (int s = 0; s < solutionCount; s++) {
        cout << "\nSolution " << s + 1 << ":" << endl;
```

```
            for (int r = 0; r < 9; r++) {
                for (int c = 0; c < 9; c++) {
                    cout << solutions[s][r * 9 + c] << " ";

                    if (c == 2 || c == 5)
                        cout << "| ";
                }

                cout << endl;

                if (r == 2 || r == 5)
                    cout << "------+-------+------" << endl;
            }
        }
    }
};

int main() {
    string filenames[] = {
        "puzzle1.txt",
        "puzzle2.txt",
        "puzzle3.txt",
        "puzzle4.txt",
        "puzzle5.txt"
    };

    // Process all five puzzle files.
    for (int i = 0; i < 5; i++) {
        cout << "\n================================" << endl;
        cout << "Puzzle file: " << filenames[i] << endl;
        cout << "================================" << endl;

        SudokuSolver solver;

        if (solver.loadPuzzle(filenames[i])) {
            cout << "\nOriginal puzzle:" << endl;
            solver.printPuzzle();

            cout << "\nResults:" << endl;
            solver.solvePuzzle();
        }
    }
```

```
    return 0;
}
```

e. Analyze the code in terms of correctness, execution time, space complexity, and maintainability as explained in the “GenAI Code Comparison” lecture.

Gemini Code:

Correctness: The correctness of this program doesn’t seem to be fully accurate, because whenever it takes in any of the given test case sudoku puzzles, it doesn’t correctly take in the input and only takes in the first line given (ex: for puzzle 1, the input that it gets is only the first row of the actual sudoku puzzle, the rest is blank), so it causes it to print every possible solution and causes an output limit since it prints out all of the 300+ possible solutions.

Execution time: The execution time for this code is exceedingly long, going over 6 seconds and additionally causing an output error, since its printing out all possible combinations for an entirely empty 9 X 9 square with numbers.

Space Complexity: The expected space complexity of the given code is at O(n^2), since the code is implementing a recursion stack with a depth of 81(9*9 square) it causes the stack frames to be large, which matches with O(n^2) space complexity.

Maintainability: The maintainability for the code is fairly good, it has good naming conventions and it encapsulates functions well so they aren’t interfering with any other parts of the code. But some issues include hardcoded inputs and outputs as well as a hardcoded file batch in the main.

Chat GPT Code:

Correctness: Unlike the Gemini code, this code successfully runs when given the text file and produces the correct answer. There aren’t any runtime errors and it shows the accurate answer compared to the given answer.

Execution time:

The execution time for the code is at most O(9^k), where k is the number of empty cells that are given in the sudoku puzzle. Since it's a brute force method, the code will check each possible method in order to see if a certain pattern matches or not. Because of this, it has to print every possible solution that it finds until it finds all possible solutions.

Space Complexity:

The overall space complexity of the program is O(1+S), since the grid and the recursive stack, so the only part that would grow is the total solutions that can be found. So overall, it's fairly optimal and there might not be that many big changes to that aspect of the code.

Maintainability:

Fairly organized, since the code uses SudokuSolver as the main way to sort the code as well as the different processes are split up pretty well among all the different functions. The main thing that could be added is human input for the file and finding better search optimizations.

f. Select one to use as the basis for this assignment and justify why you picked it.

I will use the code given by Chat GPT, since the code actually works unlike my Gemini code, so I have a lot more to work with and a lot less features to optimize since the OpenAI code is more indepth.
g. Explain what you did to improve it in terms of correctness, execution time, space complexity, and maintainability.

Correctness: Firstly I changed ValidateInitialPuzzle, which is now used to detect any duplicate digits in a row, column or block when it's searching through the puzzle. I also added error checking so that the input doesn't accept anything outside of the numbers 1-9 as well as any puzzle that isn't the exact 9*9 cube that all sudoku puzzles follow the structure of. I also made it so that solveRecursive() preserves backtracking, so that it's easier to try alternative solutions while testing possible solutions.

Execution Time: The main improvement added is focusing on checking the minimum remaining values of each cube. Essentially, the program prioritizes values with the least amount of options that It could have, which should slightly reduce the runtime of the code, despite the fact that It still runs with exponential runtime in the worst case scenario.

Space Complexity: Instead of storing each completed grid in the code, I modified it so that it doesn't need to store every completed grid simultaneously, which significantly reduces the amount of memory that is required to store from this program.

Maintainability: I added named constants, descriptive methods, as well as make a clear and distinct separation of responsibilities that the code has when trying to solve the puzzle(Ex: the function isValid() handles the sudoku rules, while findBestCell() is responsible for choosing what the next best cell).